

11 Exposing pods with services

This chapter covers

- Communication between pods
- Distributing client connections over a group of pods that provide the same service
- Discovering services in the cluster through DNS and environment variables
- Exposing services to clients outside the cluster
- Using readiness probes to add or remove individual pods from services

Instead of running a single pod to provide a particular service, people nowadays typically run several replicas of the pod so that the load can be distributed across multiple cluster nodes. However, that means all pod replicas providing the same service should be reachable at a single address so clients can use that single address, rather than having to keep track of and connect directly to individual pod instances. In Kubernetes, this is done by using Service objects.

The Kiada suite you're building consists of three services: the Kiada service, the Quiz service, and the Quote service. So far, these have been three isolated services that you interact with individually, but the plan is to connect them, as shown in figure 11.1.

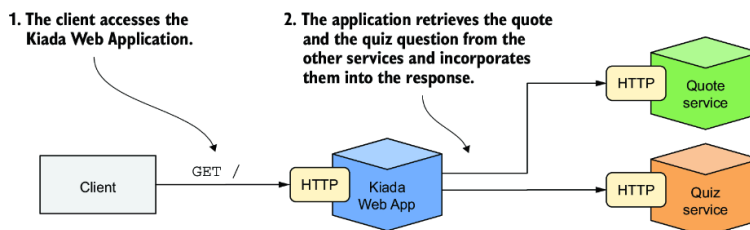


Figure 11.1 The architecture and operation of the Kiada suite

The Kiada service will call the other two services and integrate the information they return into the response it will send to the client. Multiple pod replicas will provide each service, so you'll need to use Service objects to expose them.

NOTE You'll find the code files for this chapter at <https://github.com/luksa/kubernetes-in-action-2nd-edition/tree/master/Chapter11>.

Before you create Service objects, deploy the pods and the other objects by applying the manifests in the `Chapter11/SETUP/` directory as follows:

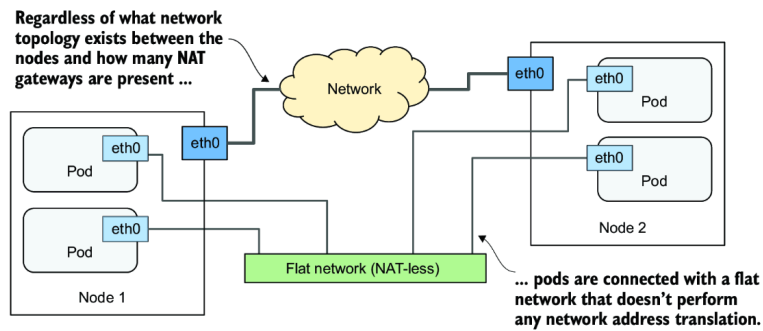
```
$ kubectl apply -f SETUP/ --recursive
```

You may recall from the previous chapter that this command applies all manifests in the specified directory and its subdirectories. After applying these manifests, you should have multiple pods in your current Kubernetes namespace.

UNDERSTANDING HOW PODS COMMUNICATE

You learned in chapter 5 what pods are, when to combine multiple containers into a pod, and how those containers communicate. But how do containers from different pods communicate?

Each pod has its own network interface with its own IP address. All pods in the cluster are connected by a single private network with a flat address space. As shown in the next figure, even if the nodes hosting the pods are geographically dispersed with many network routers in between, the pods can communicate over their own flat network where no NAT (Network Address Translation) is required. This pod network is typically a software-defined network that's layered on top of the actual network that connects the nodes.



Pods communicate via their own computer network.

When a pod sends a network packet to another pod, neither SNAT (Source NAT) nor DNAT (Destination NAT) is performed on the packet. This means that the source IP and port, and the destination IP and port, of packets exchanged directly between pods are never changed. If the sending pod knows the IP address of the receiving pod, it can send packets to it. The receiving pod can see the sender's IP as the source IP address of the packet.

Although there are many Kubernetes network plugins, they must all behave as previously described. Therefore, the communication between two pods is always the same, regardless of whether the pods are running on the same node or on nodes located in different geographic regions. The containers in the pods can communicate over the flat NAT-less network, like computers on a local area network (LAN) connected to a single network switch. From the perspective of the applications, the actual network topology between the nodes isn't important.

11.1 Exposing pods via services

If an application running in one pod needs to connect to another application running in a different pod, it needs to know the address of the other pod. This is easier said than done for the following reasons:

- Pods are *ephemeral*. A pod can be removed and replaced with a new one at any time. This happens when the pod is evicted from a node to make room for other pods, when the node fails, when the pod is no longer needed because a smaller number of pod replicas can handle the load, and for many other reasons.
- A pod gets its IP address when it's assigned to a node. You don't know the IP address of the pod in advance, so you can't provide it to the pods that will connect to it.
- In horizontal scaling, multiple pod replicas provide the same service. Each of the replicas has its own IP address. If another pod needs to connect to these replicas, it should be able to do so using a single IP or

DNS name that points to a load balancer that distributes the load across all replicas.

Also, some pods need to be exposed to clients outside the cluster. Until now, whenever you wanted to connect to an application running in a pod, you used port forwarding, which is for development only. The right way to make a group of pods externally accessible is to use a Kubernetes Service.

11.1.1 Introducing services

A Kubernetes Service is an object you create to provide a single, stable access point to a set of pods that provide the same service. Each service has a stable IP address that doesn't change for as long as the service exists. Clients open connections to that IP address on one of the exposed network ports, and those connections are then forwarded to one of the pods that back that service. In this way, clients don't need to know the addresses of the individual pods providing the service, so those pods can be scaled out or in and moved from one cluster node to another at will. A service acts as a load balancer in front of those pods.

UNDERSTANDING WHY YOU NEED SERVICES

The Kiada suite is an excellent example to explain services. It contains three sets of pods that provide three different services. The Kiada service calls the Quote service to retrieve a quote from the book, and the Quiz service to retrieve a quiz question.

I've made the necessary changes to the Kiada application in version 0.5. You can find the updated source code in directory `Chapter11/` in the book's code repository. You'll use this new version throughout this chapter. You'll learn how to configure the Kiada application to connect to the other two services, and you'll make it visible to the outside world. Since both the number of pods in each service and their IP addresses can change, you'll expose them via Service objects, as shown in figure 11.2.

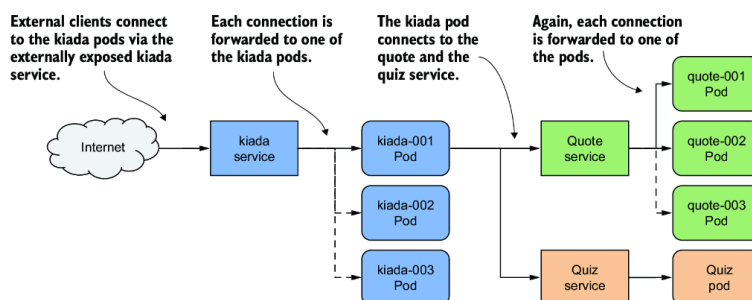


Figure 11.2 Exposing pods with Service objects

By creating a service for the Kiada pods and configuring it to be reachable from outside the cluster, you create a single, constant IP address through which external clients can connect to the pods. Each connection is forwarded to one of the Kiada pods. By creating a service for the Quote pods, you create a stable IP address through which the Kiada pods can reach the Quote pods, regardless of the number of pod instances behind the service and their location at any given time.

Although there's only one instance of the Quiz pod, it too must be exposed through a service, since the pod's IP address changes every time the pod is deleted and recreated. Without a service, you'd have to reconfigure the Kiada pods each time or make the pods get the Quiz pod's IP from the Kubernetes API. If you use a service, you don't have to do that because its IP address never changes.

UNDERSTANDING HOW PODS BECOME PART OF A SERVICE

A service can be backed by more than one pod. When you connect to a service, the connection is passed to one of the backing pods. But how do you define which pods are part of the service and which aren't?

In the previous chapter, you learned about labels and label selectors and how they're used to organize a set of objects into subsets. Services use the same mechanism. As shown in figure 11.3, you add labels to Pod objects and specify the label selector in the Service object. The pods whose labels match the selector are part of the service.

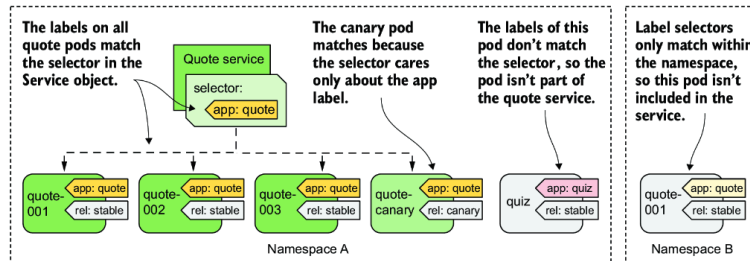


Figure 11.3 Label selectors determine which pods are part of the service.

The label selector defined in the `quote` service is `app=quote`, which means that it selects all `quote` pods, both stable and canary instances, since they all contain the label key `app` with the value `quote`. Other labels on the pods don't matter.

11.1.2 Creating and updating services

Kubernetes supports several types of services: `ClusterIP`, `NodePort`, `LoadBalancer`, and `ExternalName`. The `ClusterIP` type, which you'll learn about first, is only used internally, within the cluster. If you create a Service object without specifying its type, that's the type of service you get. The services for the Quiz and Quote pods are of this type because they're used by the Kiada pods within the cluster. The service for the Kiada pods, however, must also be accessible to the outside world, so the `ClusterIP` type isn't sufficient.

CREATING A SERVICE YAML MANIFEST

The following listing shows the minimal YAML manifest for the `quote` Service object.

Listing 11.1 YAML manifest for the `quote` Service object

```
apiVersion: v1      #1
kind: Service        #1
metadata:
  name: quote        #2
spec:
  type: ClusterIP     #3
  selector:           #4
    app: quote        #4
  ports:             #5
  - name: http        #5
    port: 80          #5
    targetPort: 80    #5
    protocol: TCP      #5
```

#1 This manifest describes a Service object.

#2 The name of this service

#3 ClusterIP services are accessible only within the cluster.

#4 The label selector that specifies which pods are part of this service

#5 Port 80 of this service is mapped to port 80 in the pods that back this service.

NOTE Since the `quote` Service object is one of the objects that make up the Quote application, you could also add the `app: quote` label to this object. However, because this label isn't required for the service to function, it's omitted in this example.

NOTE If you create a service with multiple ports, you must specify a name for each port. It's best to do the same for services with a single port.

NOTE Instead of specifying the port number in the `targetPort` field, you can also specify the name of the port as defined in the container's port list in the pod definition. This allows the service to use the correct target port number even if the pods behind the service use different port numbers.

The manifest defines a `ClusterIP` Service named `quote`. The service accepts connections on port 80 and forwards each connection to port 80 of a randomly selected pod matching the `app=quote` label selector, as shown in figure 11.4.

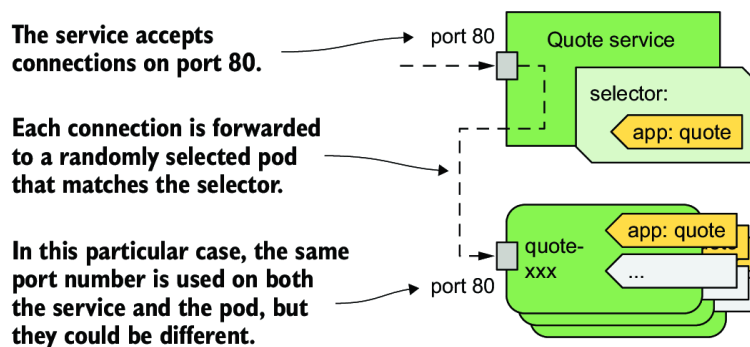


Figure 11.4 The `quote` service and the pods that it forwards traffic to

To create the service, apply the manifest file to the Kubernetes API using `kubectl apply`.

CREATING A SERVICE WITH KUBECTL EXPOSE

Normally, you create services like you create other objects, by applying an object manifest using `kubectl apply`. However, you can also create services using the `kubectl expose` command, like we did in chapter 3.

Create the service for the Quiz pod as follows:

```
$ kubectl expose pod quiz --name quiz
service/quiz exposed
```

This command creates a service named `quiz` that exposes the `quiz` pod. To do this, it checks the pod's labels and creates a Service object with a label selector that matches all the pod's labels.

NOTE In chapter 3, you used the `kubectl expose` command to expose a Deployment object. In this case, the command took the selector from the Deployment and used it in the Service object to expose all its pods. You'll learn more about Deployments in chapter 15.

You've now created two services. Section 11.1.3 describes how to connect them, but first let's see if they're configured correctly.

LISTING SERVICES

When you create a service, it's assigned an internal IP address that any workload running in the cluster can use to connect to the pods that are part of that service. This is the cluster IP address of the service. You can see it by listing services with the `kubectl get services` command. If you want to see the label selector of each service, use the `-o wide` option as follows:

```
$ kubectl get svc -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
quiz	ClusterIP	10.96.136.190	<none>	8080/TCP	15s	app=quiz, rel=stable
quote	ClusterIP	10.96.74.151	<none>	80/TCP	23s	app=quote

NOTE The shorthand for `services` is `svc`.

The output of the command shows the two services you created. For each service, the type, IP addresses, exposed ports, and label selector are printed.

NOTE You can also view the details of each service with the `kubectl describe svc` command.

You'll notice that the `quiz` service uses a label selector that selects pods with the labels `app: quiz` and `rel: stable`. This is because these are the labels of the `quiz` pod from which the service was created using the `kubectl expose` command.

Let's think about this. Do you want the `quiz` service to include only the `stable` pods? Probably not. Maybe later you decide to deploy a canary release of the `quiz` service in parallel with the `stable` version. In that case, you want traffic to be directed to both pods.

Another thing I don't like about the `quiz` service is the port number. Since the service uses HTTP, I'd prefer it to use port 80 instead of 8080. Fortunately, you can change the service after you create it.

CHANGING THE SERVICE'S LABEL SELECTOR

To change the label selector of a service, you can use the `kubectl set selector` command. To fix the selector of the `quiz` service, run the following command:

```
$ kubectl set selector service quiz app=quiz
service/quiz selector updated
```

List the services again with the `-o wide` option to confirm the selector change. This method of changing the selector is useful if you're deploying multiple versions of an application and want to redirect clients from one version to another.

CHANGING THE PORTS EXPOSED BY THE SERVICE

To change the ports that the service forwards to pods, you can edit the Service object with the `kubectl edit` command or update the manifest file and then apply it to the cluster. Before continuing, run `kubectl edit svc quiz` and change the port from `8080` to `80`, making sure to only change the `port` field and leaving the `targetPort` set to `8080`, as this is the port that the `quiz` pod listens on.

CONFIGURING BASIC SERVICE PROPERTIES

Table 11.1 lists the basic fields you can set in the Service object. Other fields are explained throughout the remainder of this chapter.

Table 11.1 Fields in the Service object’s spec for configuring the service’s basic properties

Field	Field type	Description
type	string	Specifies the type of this Service object. Allowed values are <code>ClusterIP</code> , <code>NodePort</code> , <code>LoadBalancer</code> , and <code>ExternalName</code> . The default value is <code>ClusterIP</code> . The differences between these types are explained in the following sections of this chapter.
clusterIP	string	The internal IP address within the cluster where the service is available. Normally, you leave this field blank and let Kubernetes assign the IP. If you set it to <code>None</code> , the service is a headless service. These are explained in section 11.4.
selector	map[string]string	Specifies the label keys and values that the pod must have in order for this service to forward traffic to it. If you don’t set this field, you are responsible for managing the service endpoints. This is explained in section 11.3.
ports	[]Object	List of ports exposed by this service. Each entry can specify the <code>name</code> , <code>protocol</code> , <code>appProtocol</code> , <code>port</code> , <code>nodePort</code> , and <code>targetPort</code> .

Kubernetes supports both IPv4 and IPv6. When you create a Service object, you can specify whether you want the service to be a single- or dual-stack service through the `ipFamilyPolicy` field. The default value is `SingleStack`, which means that only a single IP family is assigned to the service, regardless of whether the cluster is configured for single-stack or dual-stack networking. Set the value to `PreferDualStack` if you want the service to receive both IP families when the cluster supports dual-stack, and one IP family when it supports single-stack networking. If your service requires both an IPv4 and an IPv6 address, set the value to `RequireDualStack`. The creation of the service will be successful only on dual-stack clusters.

After you create the Service object, its `spec.ipFamilies` array indicates which IP families have been assigned to it. The two valid values are `IPv4` and `IPv6`. You can also set this field yourself to specify which IP family to assign to the service in clusters that provide dual-stack networking. The `ipFamilyPolicy` must be set accordingly or the creation will fail.

For dual-stack services, the `spec.clusterIP` field contains only one of the IP addresses, but the `spec.clusterIPs` field contains both the IPv4 and IPv6 addresses. The order of the IPs in the `clusterIPs` field corresponds to the order in the `ipFamilies` field.

11.1.3 Accessing cluster-internal services

The `ClusterIP` services you created in the previous section are accessible only within the cluster, from other pods and from the cluster nodes. You can't access them from your own machine. To check whether a service is actually working, you must either log in to one of the nodes with `ssh` and connect to the service from there, or use the `kubectl exec` command to run a command such as `curl` in an existing pod and get it to connect to the service.

NOTE You can also use the `kubectl port-forward svc/my-service` command to connect to one of the pods backing the service. However, this command doesn't connect to the service. It only uses the Service object to find a pod to connect to. The connection is then made directly to the pod, bypassing the service.

CONNECTING TO SERVICES FROM PODS

To use the service from a pod, run a shell in the `quote-001` pod as follows:

```
$ kubectl exec -it quote-001 -c nginx -- sh
/ #
```

Now check whether you can access the two services. Use the cluster IP addresses of the services that `kubectl get services` displays. In my case, the `quiz` service uses cluster IP `10.96.136.190`, whereas the `quote` service uses IP `10.96.74.151`. From the `quote-001` pod, I can connect to the two services as follows:

```
/ # curl http://10.96.136.190      #1
This is the quiz service running in pod quiz

/ # curl http://10.96.74.151      #2
This is the quote service running in pod quote-canary
```

#1 This is the cluster IP of the quiz service, as shown by `kubectl get services`.

#2 This is the cluster IP of the quote service.

NOTE You don't need to specify the port in the curl command, because you set the service port to 80, which is the default for HTTP.

If you repeat the last command several times, you'll see that the service forwards the request to a different pod each time:

```
/ # while true; do curl http://10.96.74.151; done
This is the quote service running in pod quote-canary
This is the quote service running in pod quote-003
This is the quote service running in pod quote-001
...
```

The service acts as a load balancer. It distributes requests to all the pods that are behind it.

CONFIGURING SESSION AFFINITY ON SERVICES

You can configure whether the service should forward each connection to a different pod or whether it should forward all connections from the same client to the same pod. You do this via the `spec.sessionAffinity` field in the Service object. Only two types of service session affinity are supported: `None` and `ClientIP`.

The default type is `None`, which means there's no guarantee to which pod each connection will be forwarded. However, if you set the value to `ClientIP`, all connections originating from the same IP will be forwarded to the same pod. In the `spec.sessionAffinityConfig.clientIP.timeoutSeconds` field, you can specify how long the session will persist. The default value is 3 hours.

It may surprise you that Kubernetes doesn't provide cookie-based session affinity. However, considering that Kubernetes services operate at the transport layer of the OSI network model (UDP and TCP) and not at the application layer (HTTP), they don't understand HTTP cookies at all.

RESOLVING SERVICES VIA DNS

Kubernetes clusters typically run an internal DNS server that all pods in the cluster are configured to use. In most clusters, this internal DNS service is provided by CoreDNS, whereas some clusters use kube-dns. You can see which one is deployed in your cluster by listing the pods in the `kube-system` namespace.

No matter which implementation runs in your cluster, it allows pods to resolve the cluster IP address of a service by name. Using the cluster DNS, pods can therefore connect to the `quiz` service like this:

```
/ # curl http://quiz      #1
This is the quiz service running in pod quiz
```

#1 The service name is used instead of its cluster IP.

A pod can resolve any service defined in the same namespace as the pod by simply pointing to the name of the service in the URL. If a pod needs to connect to a service in a different namespace, it must append the namespace of the Service object to the URL. For example, to connect to the `quiz` service in the `kiada` namespace, a pod can use the URL `http://quiz.kiada/` regardless of which namespace it's in.

From the `quote-001` pod where you ran the shell command, you can also connect to the service as follows:

```
/ # curl http://quiz.kiada      #1
This is the quiz service running in pod quiz
```

#1 The name of the service is quiz; kiada is the namespace.

A service is resolvable under the following DNS names:

- `<service-name>` , if the service is in the same namespace as the pod performing the DNS lookup,
- `<service-name>.<service-namespace>` from any namespace, but also under
- `<service-name>.<service-namespace>.svc` , and
- `<service-name>.<service-namespace>.svc.cluster.local` .

NOTE The default domain suffix is `cluster.local` but can be changed at the cluster level.

The reason you don't need to specify the fully qualified domain name (FQDN) when resolving the service through DNS is because of the `search` line in the pod's `/etc/resolv.conf` file. For the `quote-001` pod, the file looks like this:

```
/ # cat /etc/resolv.conf
search kiada.svc.cluster.local svc.cluster.local cluster.local localdomain
nameserver 10.96.0.10
options ndots:5
```

When you try to resolve a service, the domain names specified in the `search` field are appended to the name until a match is found. If you're wondering what the IP address is in the `nameserver` line, you can list all the services in your cluster to find out:

```
$ kubectl get svc -A
```

NAMESPACE	NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
default	kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
kiada	quiz	ClusterIP	10.96.136.190	<none>	80/TCP
kiada	quote	ClusterIP	10.96.74.151	<none>	80/TCP
kube-system	kube-dns	ClusterIP	10.96.0.10	<none>	53/UDP... #1

#1 Here's the IP address you're looking for.

The `nameserver` in the pod's `resolv.conf` file points to the `kube-dns` service in the `kube-system` namespace. This is the cluster DNS service that the pods use. As an exercise, try to figure out which pod(s) this service forwards traffic to.

CONFIGURING THE POD'S DNS POLICY

Whether or not a pod uses the internal DNS server can be configured using the `dnsPolicy` field in the pod's spec. The default value is `ClusterFirst`, which means that the pod uses the internal DNS first and then the DNS configured for the cluster node. Other valid values are `Default` (uses the DNS configured for the node), `None` (no DNS configuration is provided by Kubernetes; you must configure the pod's DNS settings using the `dnsConfig` field explained in the next paragraph), and `ClusterFirstWithHostNet` (for special pods that use the host's network instead of their own, which is explained later in the book).

Setting the `dnsPolicy` field affects how Kubernetes configures the pod's `resolv.conf` file. You can further customize this file through the pod's `dnsConfig` field. The `pod-with-dns-options.yaml` file in the book's code repository demonstrates the use of this field.

DISCOVERING SERVICES THROUGH ENVIRONMENT VARIABLES

Nowadays, virtually every Kubernetes cluster offers the cluster DNS service. In the early days, this wasn't the case. Back then, the pods found the IP addresses of the services using environment variables. These variables still exist today.

When a container is started, Kubernetes initializes a set of environment variables for each service that exists in the pod's namespace. Let's see what these environment variables look like by exploring the environment of one of your running pods. Since you created your pods before the services, you won't see any environment variables related to the services except those for the `kubernetes` service, which exists in the `default` namespace.

NOTE The `kubernetes` service forwards traffic to the Kubernetes API server.

To see the environment variables for the two services that you created, you must restart the container as follows:

```
$ kubectl exec quote-001 -c nginx -- kill 1
```

When the container is restarted, its environment variables contain the entries for the `quiz` and `quote` services. Display them with the following command:

```
$ kubectl exec -it quote-001 -c nginx -- env | sort
...
QUIZ_PORT_80_TCP_ADDR=10.96.136.190      #1
QUIZ_PORT_80_TCP_PORT=80                 #1
QUIZ_PORT_80_TCP_PROTO=tcp              #1
QUIZ_PORT_80_TCP=tcp://10.96.136.190:80  #1
QUIZ_PORT=tcp://10.96.136.190:80         #1
QUIZ_SERVICE_HOST=10.96.136.190          #1
QUIZ_SERVICE_PORT=80                     #1
QUOTE_PORT_80_TCP_ADDR=10.96.74.151      #2
QUOTE_PORT_80_TCP_PORT=80                 #2
QUOTE_PORT_80_TCP_PROTO=tcp              #2
QUOTE_PORT_80_TCP=tcp://10.96.74.151:80  #2
QUOTE_PORT=tcp://10.96.74.151:80         #2
QUOTE_SERVICE_HOST=10.96.74.151          #2
QUOTE_SERVICE_PORT=80                     #2
```

#1 The environment variables describing the quiz service

#2 The environment variables describing the quote service

Quite a handful of environment variables, wouldn't you say? For services with multiple ports, the number of variables is even larger. An application running in a container can use these variables to find the IP address and port(s) of a particular service.

NOTE In the environment variable names, the hyphens in the service name are converted to underscores and all letters are uppercased.

Nowadays, applications usually get this information through DNS, so these environment variables aren't as useful as in the early days. They can even cause problems. If the number of services in a namespace is too large, any pod you create in that namespace will fail to start. The container exits with exit code 1, and you see the following error message in the container's log:

```
standard_init_linux.go:228: exec user process caused: argument list too long
```

To prevent this, you can disable the injection of service information into the environment by setting the `enableServiceLinks` field in the pod's spec to `false`.

UNDERSTANDING WHY YOU CAN'T PING A SERVICE IP

You've learned how to verify that a service is forwarding traffic to your pods. But what if it doesn't? In that case, you might want to try pinging the service's IP. Why don't you try that right now? Ping the `quiz` service from the `quote-001` pod as follows:

```
$ kubectl exec -it quote-001 -c nginx -- ping quiz
PING quiz (10.96.136.190): 56 data bytes
^C
--- quiz ping statistics ---
15 packets transmitted, 0 packets received, 100% packet loss
command terminated with exit code 1
```

Wait a few seconds and then interrupt the process by pressing Ctrl-C. As you can see, the IP address was resolved correctly, but none of the packets got through. This is because the IP address of the service is virtual and has meaning only in conjunction with one of the ports defined in the service. You can't ping a service's IP address.

USING SERVICES IN A POD

Now that you know that the Quiz and Quote services are accessible from pods, you can deploy the Kiada pods and configure them to use the two services. The application expects the URLs of these services in the environment variables `QUIZ_URL` and `QUOTE_URL`. These aren't environment variables that Kubernetes adds on its own, but variables that you set manually so that the application knows where to find the two services. Therefore, the `env` field of the `kiada` container must be configured as in the following listing.

Listing 11.2 Configuring the service URLs in the Kiada pod

```
...
  env:
    - name: QUOTE_URL                #1
      value: http://quote/quote      #1
    - name: QUIZ_URL                #2
      value: http://quiz             #2
    - name: POD_NAME
  ....
```

#1 The URL where the Quote service returns a quote from the book

#2 The base URL of the Quiz service

The environment variable `QUOTE_URL` is set to `http://quote/quote`. The hostname is the same as the name of the service you created in the previous section. Similarly, `QUIZ_URL` is set to `http://quiz`, where `quiz` is the name of the other service you created.

Deploy the Kiada pods by applying the manifest file `kiada-stable-and-canary.yaml` to your cluster using `kubectl apply`. Then run the following command to open a tunnel to one of the pods you just created:

```
$ kubectl port-forward kiada-001 8080 8443
```

You can now test the application at `http://localhost:8080` or `https://localhost:8443`. If you use `curl`, you should see a response like the following:

```
$ curl http://localhost:8080
```

```
==== TIP OF THE MINUTE
```

Kubectl options that take a value can be specified with an equal sign or with a space

```
==== POP QUIZ
```

```
First question
```

```
0) First answer
```

```
1) Second answer
```

```
2) Third answer
```

Submit your answer to `/question/1/answers/<index of answer>` using the POST method.

```
==== REQUEST INFO
```

Request processed by Kobia 1.0 running in pod "kiada-001" on node "kind-worker2".

Pod hostname: kiada-001; Pod IP: 10.244.1.90; Node IP: 172.18.0.2; Client IP: ::ffff

HTML version of this content is available at `/html`

If you open the URL in your web browser, you get the web page shown in figure 11.5.

Displays a quote from the book. The quote is fetched from the quote service.

Presents a multiple-choice Kubernetes-related question. The data is fetched from the quiz service.



Figure 11.5 The Kiada application when accessed with a web browser

If you can see the quote and quiz question, it means that the `kiada-001` pod is able to communicate with the `quote` and `quiz` services. If you check the logs of the pods that back these services, you'll see that they are receiving requests. In the case of the `quote` service, which is backed by multiple pods, you'll see that each request is sent to a different pod.

11.2 Exposing services externally

ClusterIP services like the ones you created in the previous section are only accessible within the cluster. Because clients must be able to access the Kiada service from outside the cluster, as shown in figure 11.6, creating a ClusterIP service won't suffice.

The kiada service must be reachable from outside the cluster.

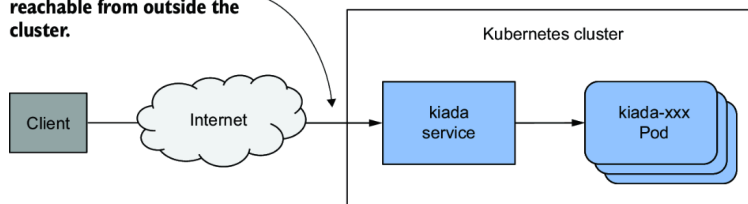


Figure 11.6 Exposing a service externally

If you need to make a service available to the outside world, you can do one of the following:

- Assign an additional IP to a node and set it as one of the service's `externalIPs`
- Set the service's type to `NodePort` and access the service through the node's port(s)
- Ask Kubernetes to provision a load balancer by setting the type to `LoadBalancer`
- Expose the service through an Ingress object

A rarely used method is to specify an additional IP in the `spec.externalIPs` field of the Service object. By doing this, you're telling Kubernetes to treat any traffic directed to that IP address as traffic to be processed by the service. When you ensure that this traffic arrives at a node with the service's external IP as its destination, Kubernetes forwards it to one of the pods that back the service.

A more common way to make a service available externally is to set its type to `NodePort`. Kubernetes makes the service available on a network port on all cluster nodes (the so-called node port, from which this service type gets its name). Like `ClusterIP` services, the service gets an internal cluster IP, but it is also accessible through the node port on each of the cluster nodes. Usually, you then provision an external load balancer that redirects traffic to these node ports. The clients can connect to your service via the load balancer's IP address.

Instead of using a `NodePort` service and manually setting up the load balancer, Kubernetes can also do this for you if you set the service type to `LoadBalancer`. However, not all clusters support this service type, as the provisioning of the load balancer depends on the infrastructure the cluster is running on. Most cloud providers support `LoadBalancer` services in their clusters, whereas clusters deployed on premises require an add-on such as MetalLB, a load-balancer implementation for bare-metal Kubernetes clusters.

The final way to expose a group of pods externally is radically different. Instead of exposing the service externally via node ports and load balancers, you can use an Ingress object. How this object exposes the service depends on the underlying ingress controller, but it allows you to expose many services through a single externally reachable IP address. You'll learn more about this in the next chapter.

11.2.1 Exposing pods through a NodePort service

One way to make pods accessible to external clients is to expose them through a `NodePort` service. When you create such a service, the pods that match its selector are accessible through a specific port on all nodes in the cluster, as shown in figure 11.7. Because this port is open on the nodes, it's called a node port.

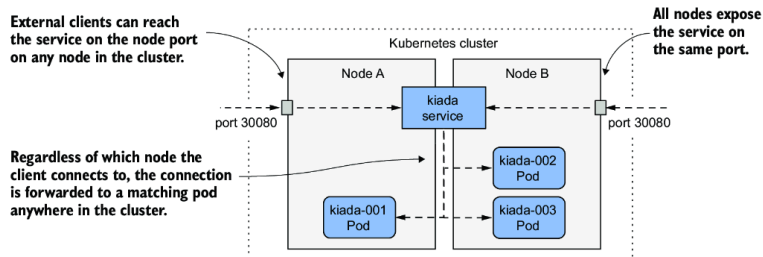


Figure 11.7 Exposing pods through a `NodePort` service

Like a `ClusterIP` service, a `NodePort` service is accessible through its internal cluster IP, but also through the node port on each of the cluster nodes. In the example shown in the figure, the pods are accessible through port `30080`. As you can see, this port is open on both cluster nodes.

It doesn't matter which node a client connects to because all the nodes will forward the connection to a pod that belongs to the service, regardless of which node is running the pod. When the client connects to node A, a pod on either node A or B can receive the connection. The same is true when the client connects to the port on node B.

CREATING A NODEPORT SERVICE

To expose the Kiada pods through a `NodePort` service, you create the service from the manifest shown in the following listing. You can find it in the file `svc.kiada .nodeport.yaml`.

Listing 11.3 A NodePort service exposing the Kiada pods on two ports

```
apiVersion: v1
kind: Service
metadata:
  name: kiada
spec:
  type: NodePort      #1
  selector:
    app: kiada
  ports:
    - name: http      #2
      port: 80         #3
      nodePort: 30080  #4
      targetPort: 8080 #5
    - name: https     #6
      port: 443        #6
      nodePort: 30443  #6
      targetPort: 8443 #6
```

#1 The service type is NodePort.

#2 The service exposes two ports. This here is the HTTP port.

#3 The port on the service's cluster IP.

#4 The service is accessible through port 30080 of each of your cluster nodes.

#5 This is the port that the pods listen on.

#6 The service exposes another port for HTTPS.

Compared to the `ClusterIP` services you created earlier, the type of service in the listing is `NodePort`. Unlike the previous services, this service exposes two ports and defines the `nodePort` numbers for each of those ports.

NOTE You can omit the `nodePort` field to allow Kubernetes to assign the port number. This prevents port conflicts between different NodePort services.

The service specifies six different port numbers, which might make it difficult to understand, but figure 11.8 should help you make sense of it.

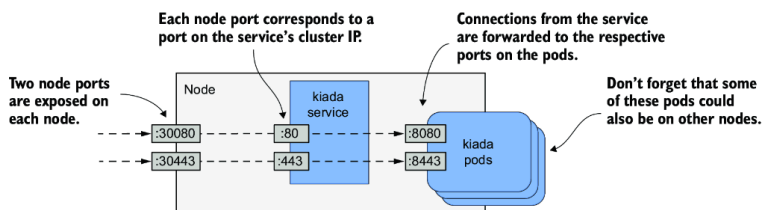


Figure 11.8 Exposing multiple ports through with a NodePort service

EXAMINING YOUR NODEPORT SERVICE

After you create the service, inspect it with the `kubectl get` command as follows:

```
$ kubectl get svc
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kiada	NodePort	10.96.226.212	<none>	80:30080/TCP,443:30443/TCP	1m #
quiz	ClusterIP	10.96.173.186	<none>	80/TCP	3h
quote	ClusterIP	10.96.161.97	<none>	80/TCP	3h

#1 This is the service you created.

Compare the `TYPE` and `PORT(S)` columns of the services you've created so far. Unlike the two `ClusterIP` services, the `kiada` service is a `NodePort` service that exposes node ports 30080 and 30443 in addition to ports 80 and 443 available on the service's cluster IP.

ACCESSING A NODEPORT SERVICE

To find out all `IP:port` combinations over which the service is available, you need not only the node port number(s), but also the IPs of the nodes. You can get these by running `kubectl get nodes -o wide` and looking at the `INTERNAL-IP` and `EXTERNAL-IP` columns. Clusters running in the cloud usually have the external IP set for the nodes, whereas clusters running on bare metal may set only the internal IP of the nodes. You should be able to reach the node ports using these IPs, if there are no firewalls in the way.

NOTE To allow traffic to node ports when using GKE, run `gcloud compute firewall-rules create gke-allow-nodeports --allow=tcp:30000-32767`. If your cluster is running on a different cloud provider, check the provider's documentation on how to configure the firewall to allow access to node ports.

In the cluster I provisioned with the `kind` tool, the internal IPs of the nodes are as follows:

```
$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	...	INTERNAL-IP	EXTERNAL-IP
kind-control-plane	Ready	control-plane,master	...	172.18.0.3	<none>
kind-worker	Ready	<none>	...	172.18.0.4	<none>
kind-worker2	Ready	<none>	...	172.18.0.2	<none>

The `kiada` service is available on all these IPs, even the IP of the node running the Kubernetes control plane. I can access the service at any of the following URLs:

- `10.96.226.212:80` within the cluster (this is the cluster IP and the internal port).
- `172.18.0.3:30080` from wherever the node `kind-control-plane` is reachable, as this is the node's IP address; the port is one of the node ports of the `kiada` service.
- `172.18.0.4:30080` (the second node's IP address and the node port).
- `172.18.0.2:30080` (the third node's IP address and the node port).

NOTE On macOS, the cluster node(s) may not be reachable from the host OS. Refer to the documentation of the tool you used to deploy the Kubernetes cluster to see if a workaround exists.

TIP If you're unable to access the service via the node ports, check whether it's possible via its internal cluster IP from within the cluster, as described previously.

The service is also accessible via HTTPS on port `443` within the cluster and via node port `30443`. If my nodes also had external IPs, the service would also be available through the two node ports on those IPs. If you're using Minikube or another single-node cluster, you should use the IP of that node.

TIP If you're using Minikube, you can easily access your `NodePort` services through your browser by running `minikube service <service-name> [-n <namespace>]`.

Use `curl` or your web browser to access the service. Select one of the nodes and find its IP address. Send the HTTP request to port `30080` of this IP. Check the end of the response to see which pod handled the re-

quest and which node the pod is running on. For example, here's the response I received to one of my requests:

```
$ curl 172.18.0.4:30080
...
==== REQUEST INFO
Request processed by Kuba 1.0 running in pod "kiada-001" on node "kind-worker2".
Pod hostname: kiada-001; Pod IP: 10.244.1.90; Node IP: 172.18.0.2; Client IP: ::ffff
```

Notice that I sent the request to the 172.18.0.4, which is the IP of the kind-worker node, but the pod that handled the request was running on the node kind-worker2. The first node forwarded the connection to the second node, as explained in the introduction to NodePort services.

Did you also notice where the pod thought the request came from? Look at the Client IP at the end of the response. That's not the IP of the computer from which I sent the request. You may have noticed that it's the IP of the node I sent the request to. I explain why this happens and how you can prevent it in section 11.2.3.

Try sending the request to the other nodes as well. You'll see that they all forward the requests to a random Kiada pod. If your nodes are reachable from the internet, the application is now accessible to users all over the world. You could use round robin DNS to distribute incoming connections across the nodes or put a proper Layer 4 load balancer in front of the nodes and point the clients to it. Or you could just let Kubernetes do this, as explained in the next section.

11.2.2 Exposing a service through an external load balancer

In the previous section, you created a service of type NodePort. Another service type is LoadBalancer. As the name suggests, this service type makes your application accessible through a load balancer. While all services act as load balancers, creating a LoadBalancer service causes an actual load balancer to be provisioned.

As shown in figure 11.9, this load balancer stands in front of the nodes and handles the connections coming from the clients. It routes each connection to the service by forwarding it to the node port on one of the nodes. This is possible because the LoadBalancer service type is an extension of the NodePort type, which makes the service accessible through these node ports. By pointing clients to the load balancer rather than directly to the node port of a particular node, the client never attempts to connect to an unavailable node because the load balancer forwards traffic only to healthy nodes. In addition, the load balancer ensures that connections are distributed evenly across all nodes in the cluster.

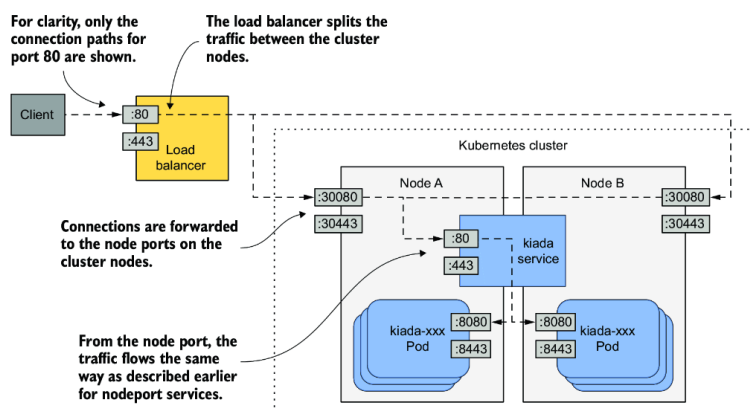


Figure 11.9 Exposing a LoadBalancer service

Not all Kubernetes clusters support this type of service, but if your cluster runs in the cloud, it almost certainly does. If your cluster runs on premises, it'll support `LoadBalancer` services if you install an add-on. If the cluster doesn't support this type of service, you can still create services of this type, but the service is only accessible through its node ports.

CREATING A LOADBALANCER SERVICE

The following listing shows the YAML manifest of a service of type `LoadBalancer` that you can find in the file `svc.kiada.loadbalancer.yaml`.

Listing 11.4 A `LoadBalancer`-type service

```
apiVersion: v1
kind: Service
metadata:
  name: kiada
spec:
  type: LoadBalancer    #1
  selector:
    app: kiada
  ports:
    - name: http
      port: 80
      nodePort: 30080
      targetPort: 8080
    - name: https
      port: 443
      nodePort: 30443
      targetPort: 8443
```

#1 Kubernetes will provision a load balancer for this service.

This manifest differs from the manifest of the `NodePort` service you deployed earlier in only one line—the line specifying the service `type`. The selector and ports are the same as before. The node ports are only specified so that they aren't randomly selected by Kubernetes. If you don't care about the node port numbers, you can omit the `nodePort` fields.

Apply the manifest with `kubectl apply`. You don't have to delete the existing `kiada` service first. This approach ensures that the internal cluster IP of the service remains unchanged.

CONNECTING TO THE SERVICE THROUGH THE LOAD BALANCER

After you create the service, it may take a few minutes for the cloud infrastructure to create the load balancer and update its IP address in the Service object. This IP address will then appear as the external IP address of your service:

```
$ kubectl get svc kiada
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
kiada	LoadBalancer	10.96.226.212	172.18.255.200	80:30080/TCP,443:30443/TCP	10m

In my case, the IP address of the load balancer is `172.18.255.200`, and I can reach the service through ports `80` and `443` of this IP. Until the load balancer is created, `<pending>` is displayed in the `EXTERNAL-IP` column instead of an IP address. This could be because the provisioning process isn't yet complete or because the cluster doesn't support `LoadBalancer` services.

NOTE If your cluster runs in a VM, as is typically the case on macOS, the load balancer IP may not be reachable from the host OS, but only from within the VM. Refer to the documentation of the tool you used to deploy the cluster to learn if there's a way to access the IP from the host. Alternatively, try accessing the IP from within the VM.

ADDING SUPPORT FOR LOADBALANCER SERVICES WITH METALLB

If your cluster runs on bare metal, you can install MetalLB to support `LoadBalancer` services. You can find it at metallb.universe.tf. If you created your cluster with the `kind` tool, you can install MetalLB using the `install-metallb-kind.sh` script from the book's code repository. If you created your cluster with another tool, you can check the MetalLB documentation for how to install it.

Adding support for LoadBalancer services is optional. You can always use the node ports directly. The load balancer is just an additional layer.

TWEAKING LOADBALANCER SERVICES

LoadBalancer services are easy to create. You just set the `type` to `LoadBalancer`. However, if you need more control over the load balancer, you can configure it with the additional fields in the Service object's spec explained in table 11.2.

Table 11.2 Fields in the service spec you can use to configure LoadBalancer services

Field	Field type	Description
loadBalancerClass	string	If the cluster supports multiple classes of load balancers, you can specify which one to use for this service. The possible values depend on the load balancer controllers installed in the cluster.
loadBalancerIP	string	If supported by the cloud provider, this field can be used to specify the desired IP for the load balancer.
loadBalancerSourceRanges	[]string	Restricts the client IPs that are allowed to access the service through the load balancer. It is not supported by all load balancer controllers.
allocateLoadBalancerNodePorts	boolean	Specifies whether to allocate node ports for this LoadBalancer - type service. Some load balancer implementations can forward traffic to pods without relying on node ports.

11.2.3 Configuring the external traffic policy for a service

You've already learned that when an external client connects to a service through the node port, either directly or through the load balancer, the connection may be forwarded to a pod that's on a different node than the one that received the connection. In this case, an additional network hop must be made to reach the pod, which results in increased latency.

Also, as mentioned earlier, when forwarding the connection from one node to another in this manner, the source IP must be replaced with the IP of the node that originally received the connection. This obscures the IP address of the client. Thus, the application running in the pod can't see where the connection is coming from. For example, a web server running in a pod can't record the true client IP in its access log.

The reason the node needs to change the source IP is to ensure that the returned packets are sent back to the node that originally received the connection so that it can return them to the client.

PROS AND CONS OF THE LOCAL EXTERNAL TRAFFIC POLICY

Both the additional network hop and the source IP obfuscation problem can be solved by preventing nodes from forwarding traffic to pods that aren't running on the same node. This is done by setting the `externalTrafficPolicy` field in the Service object's `spec` field to `Local`. This way, a node forwards external traffic only to pods running on the node that received the connection.

However, setting the external traffic policy to `Local` leads to other problems. First, if there are no local pods on the node that received the connection, the connection hangs. You must therefore ensure that the load balancer forwards connections only to nodes that have at least one such pod. This is done using the `healthCheckNodePort` field. The external load balancer uses this node port to check whether a node contains endpoints for the service. This approach allows the load balancer to forward traffic only to nodes that have such a pod.

The second problem you run into when the external traffic policy is set to `Local` is the uneven distribution of traffic across pods. If the load balancers distribute traffic evenly among the nodes, but each node runs a different number of pods, the pods on the nodes with fewer pods will receive a higher amount of traffic.

COMPARING THE CLUSTER AND THE LOCAL EXTERNAL TRAFFIC POLICIES

Consider the case presented in figure 11.10. There's one pod running on node A and two on node B. The load balancer routes half of the traffic to node A and the other half to node B.

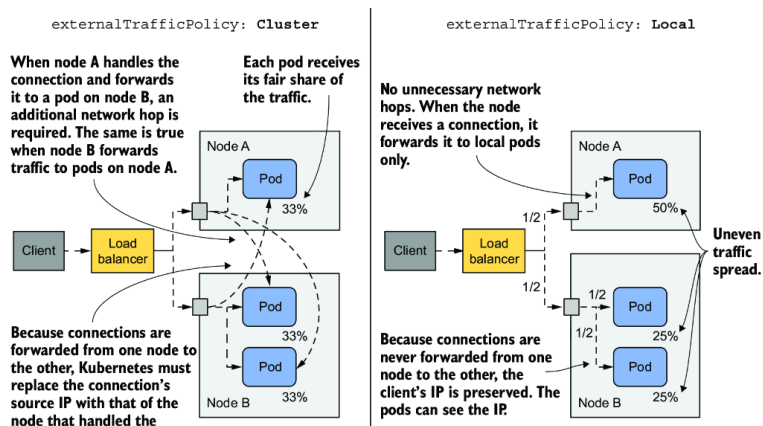


Figure 11.10 Understanding the two external traffic policies for NodePort and LoadBalancer services

When `externalTrafficPolicy` is set to `Cluster`, each node forwards traffic to all pods in the system. Traffic is split evenly between the pods. Additional network hops are required, and the client IP is obfuscated.

When the `externalTrafficPolicy` is set to `Local`, all traffic arriving at node A is forwarded to the single pod on that node. This means that this pod receives 50% of all traffic. Traffic arriving at node B is split between two pods. Each pod receives 25% of the total traffic processed by the load balancer. There are no unnecessary network hops, and the source IP is that of the client.

As with most decisions you make as an engineer, which external traffic policy to use in each service depends on what tradeoffs you're willing to make.

11.3 Managing service endpoints

So far, you've learned that services are backed by pods, but that's not always the case. The endpoints to which a service forwards traffic can be anything that has an IP address.

11.3.1 Introducing the Endpoints object

A service is typically backed by a set of pods whose labels match the label selector defined in the Service object. Apart from the label selector, the Service object's `spec` or `status` section doesn't contain the list of pods that are part of the service. However, if you use `kubectl describe` to inspect the service, you'll see the IPs of the pods under `Endpoints`, as follows:

```
$ kubectl describe svc kiada
Name:                kiada
...
Port:                http 80/TCP
TargetPort:          8080/TCP
NodePort:            http 30080/TCP
Endpoints:           10.244.1.7:8080,10.244.1.8:8080,10.244.1.9:8080 + 1
                    more... #1
...
```

#1 The list of endpoints (pod IPs and ports) for this service

The `kubectl describe` command collects this data not from the Service object, but from an Endpoints object whose name matches that of the service. The endpoints of the `kiada` service are specified in the `kiada` Endpoints object.

LISTING ENDPOINTS OBJECTS

You can retrieve Endpoints objects in the current namespace as follows:

```
$ kubectl get endpoints
NAME          ENDPOINTS                                                                                                     AGE
kiada         10.244.1.7:8443,10.244.1.8:8443,10.244.1.9:8443 + 5 more...      25m
quiz          10.244.1.11:8080                                                66m
quote         10.244.1.10:80,10.244.2.10:80,10.244.2.8:80 + 1 more...        66m
```

NOTE The shorthand for `endpoints` is `ep`. Also, the object kind is Endpoints (plural form) not Endpoint. Running `kubectl get endpoint` fails with an error.

As you can see, there are three Endpoints objects in the namespace. One for each service. Each Endpoints object contains a list of IP and port combinations that represent the endpoints for the service.

INSPECTING AN ENDPOINTS OBJECT MORE CLOSELY

To see which pods represent these endpoints, use `kubectl get -o yaml` to retrieve the full manifest of the Endpoints object as follows:

```
$ kubectl get ep kiada -o yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: kiada          #1
  namespace: kiada     #1
  ...
subsets:
- addresses:
  - ip: 10.244.1.7      #2
    nodeName: kind-worker #3
    targetRef:
      kind: Pod
      name: kiada-002    #4
      namespace: kiada  #4
      resourceVersion: "2950"
      uid: 18cea623-0818-4ff1-9fb2-cddcf5d138c3
    ...                #5
  ports:               #6
  - name: https        #6
    port: 8443         #6
    protocol: TCP      #6
  - name: http         #6
    port: 8080         #6
    protocol: TCP      #6
```

#1 The name and namespace of this Endpoints object. These always match the name and namespace of the associated Service object.

#2 The IP address of the first endpoint (a pod that matches the label selector)

#3 The name of the cluster node on which the pod runs

#4 The name and namespace of the pod

#5 The entries for other pods that match the selector are not shown.

#6 The list of ports that these endpoints expose. It matches the ports defined in the Service.

As you can see, each pod is listed as an element of the `addresses` array. In the `kiada` Endpoints object, all endpoints are in the same endpoint subset, because they all use the same port numbers. However, if one group of pods uses port `8080`, for example, and another uses port `8088`, the Endpoints object would contain two subsets, each with its own ports.

UNDERSTANDING WHO MANAGES THE ENDPOINTS OBJECT

You didn't create any of the three Endpoints objects. They were created by Kubernetes when you created the associated Service objects. These objects are fully managed by Kubernetes. Each time a new pod appears or disappears that matches the Service's label selector, Kubernetes updates the Endpoints object to add or remove the endpoint associated with the pod. You can also manage a service's endpoints manually. You'll learn how to do that later.

11.3.2 Introducing the EndpointSlice object

As you can imagine, the size of an Endpoints object becomes a problem when a service contains a very large number of endpoints. Kubernetes control plane components need to send the entire object to all cluster nodes every time a change is made. In large clusters, this leads to noticeable performance problems. To counter this, the EndpointSlice object was introduced, and the Endpoints object was deprecated. The EndpointSlice object splits the endpoints of a single service into multiple slices, improving performance and handling of those endpoints.

While an Endpoints object contains multiple endpoint subsets, each EndpointSlice contains only one. If two groups of pods expose the service on different ports, they appear in two different EndpointSlice objects. Also, an EndpointSlice object supports a maximum of 1,000 endpoints, but by default Kubernetes only adds up to 100 endpoints to each slice. The number of ports in a slice is also limited to 100. Therefore, a service with hundreds of endpoints or many ports can have multiple EndpointSlices objects associated with it.

Like Endpoints, EndpointSlices are created and managed automatically.

LISTING ENDPOINTSlices OBJECTS

In addition to the Endpoints objects, Kubernetes creates the EndpointSlice objects for your three services. You can see them with the `kubectl get endpointslices` command:

```
$ kubectl get endpointslices
```

NAME	ADDRESSTYPE	PORTS	ENDPOINTS	AG
kiada-m24zq	IPv4	8080,8443	10.244.1.7,10.244.1.8,10.244.1.9 + 1 more...	80
quiz-qbckq	IPv4	8080	10.244.1.11	79
quote-5dqhx	IPv4	80	10.244.2.8,10.244.1.10,10.244.2.9 + 1 more...	79

NOTE As of this writing, there is no shorthand for `endpointslices`.

You'll notice that unlike Endpoints objects, whose names match the names of their respective Service objects, each EndpointSlice object contains a randomly generated suffix after the service name. This way, many EndpointSlice objects can exist for each service.

LISTING ENDPOINTSlices FOR A PARTICULAR SERVICE

To see only the EndpointSlice objects associated with a particular service, you can specify a label selector in the `kubectl get` command. To list the EndpointSlice objects associated with the `kiada` service, use the label selector `kubernetes.io/service-name=kiada` as follows:

```
$ kubectl get endpointslices -l kubernetes.io/service-name=kiada
NAME                ADDRESSTYPE  PORTS                ENDPOINTS                AG
kiada-m24zq         IPv4          8080,8443           10.244.1.7,10.244.1.8,10.244.1.9 + 1 more... 88
```

INSPECTING AN ENDPOINTSLSICE

To examine an EndpointSlice object in more detail, use `kubectl describe`. Since the `describe` command doesn't require the full object name, and all EndpointSlice objects associated with a service begin with the service name, you can see them all by specifying only the service name:

```
$ kubectl describe endpointslice kiada
Name:                kiada-m24zq
Namespace:           kiada
Labels:              endpointslice.kubernetes.io/managed-by=endpointslice-controller.k8s.i
                    kubernetes.io/service-name=kiada
Annotations:         endpoints.kubernetes.io/last-change-trigger-time: 2021-10-30T08:36:21
AddressType:         IPv4
Ports:               #1
  Name    Port    Protocol    #1
  ----    -
  http    8080    TCP         #1
  https   8443    TCP         #1
Endpoints: #1
- Addresses: 10.244.1.7      #2
  Conditions:
    Ready:    true
    Hostname: <unset>
    TargetRef: Pod/kiada-002      #3
    Topology: kubernetes.io/hostname=kind-worker      #4
...

```

- #1 The ports exposed by the endpoints in this slice
- #2 The IP address of the first endpoint
- #3 The kiada-002 pod represents this service endpoint
- #4 Topology information for this endpoint. It is explained later in this chapter.

NOTE If multiple EndpointSlices match the name you provide to `kubectl describe`, the command will print all of them.

The information in the output of the `kubectl describe` command isn't much different from the information in the Endpoint object you saw earlier. The EndpointSlice object contains a list of ports and endpoint addresses, as well as information about the pods that represent those endpoints. This includes the pod's topology information, which is used for topology-aware traffic routing. You'll learn about it later in this chapter.

11.3.3 Managing service endpoints manually

When you create a Service object with a label selector, Kubernetes automatically creates and manages the Endpoints and EndpointSlice objects and uses the selector to determine the service endpoints. However, you can also manage endpoints manually by creating the Service object without a label selector. In this case, you must create the Endpoints object yourself. You don't need to create the EndpointSlice objects because Kubernetes mirrors the Endpoints object to create corresponding EndpointSlices.

Typically, you manage service endpoints this way when you want to make an existing external service accessible to pods in your cluster under a different name. This way, the service can be found through the cluster DNS and environment variables.

CREATING A SERVICE WITHOUT A LABEL SELECTOR

The following listing shows an example of a Service object manifest that doesn't define a label selector. You can find this manifest in the file `svc.external-service.yaml`. After you create the service, you'll manually configure the endpoints for it.

Listing 11.5 A service with no pod selector

```
apiVersion: v1
kind: Service
metadata:
  name: external-service    #1
spec:                       #2
  ports:                   #2
  - name: http             #2
    port: 80               #2
```

#1 The name of the service must match the name of the Endpoints object (see next listing).

#2 No label selector is defined for this service.

The manifest in the listing defines a service named `external-service` that accepts incoming connections on port `80`. As explained in the first part of this chapter, pods in the cluster can use the service either through its cluster IP address, which is assigned when you create the service, or through its DNS name.

CREATING AN ENDPOINTS OBJECT

If a service doesn't define a pod selector, no Endpoints object is automatically created for it. You must do this yourself. The following listing shows the manifest of the Endpoints object for the service you created in the previous section.

Listing 11.6 An Endpoints object created by hand

```
apiVersion: v1
kind: Endpoints
metadata:
  name: external-service    #1
subsets:
- addresses:
  - ip: 1.1.1.1            #2
  - ip: 2.2.2.2            #2
  ports:
  - name: http             #3
    port: 88               #3
```

#1 The name of the Endpoints object must match the name of the service (see previous listing).

#2 The IPs of the endpoints that the service will forward connections to

#3 The port on which the endpoints expose the service

The Endpoints object must have the same name as the service and contain the list of destination addresses and ports. In the listing, IP addresses `1.1.1.1` and `2.2.2.2` represent the endpoints for the service.

NOTE You don't have to create the EndpointSlice object. Kubernetes creates it from the Endpoints object.

The creation of the Service and its associated Endpoints object allows pods to use this service in the same way as other services defined in the cluster. As shown in figure 11.11, traffic sent to the service's cluster IP is

distributed to the service's endpoints. These endpoints are outside the cluster but could also be internal.

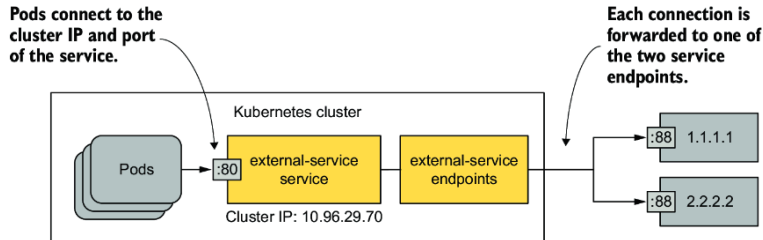


Figure 11.11 Pods consuming a service with two external endpoints

If you later decide to migrate the external service to pods running inside the Kubernetes cluster, you can add a selector to the service to redirect traffic to those pods instead of the endpoints you configured by hand. This is because Kubernetes immediately starts managing the Endpoints object after you add the selector to the service.

You can also do the opposite: if you want to migrate an existing service from the cluster to an external location, remove the selector from the Service object so that Kubernetes no longer updates the associated Endpoints object. From then on, you can manage the service's endpoints manually.

You don't have to delete the service to do this. By changing the existing Service object, the cluster IP address of the service remains constant. The clients using the service won't even notice that you've relocated the service.

11.4 Understanding DNS records for Service objects

An important aspect of Kubernetes services is the ability to look them up via DNS. This is something that needs to be looked at more closely.

You know that a service is assigned an internal cluster IP address that pods can resolve through the cluster DNS. This is because each service gets an `A` record in DNS (or an `AAAA` record for IPv6). However, a service also receives an `SRV` record for each of the ports it makes available.

Let's take a closer look at these DNS records. First, run a one-off pod like this:

```
$ kubectl run -it --rm dns-test --image=giantswarm/tiny-tools
/ #
```

This command runs a pod named `dns-test` with a container based on the container image `giantswarm/tiny-tools`. This image contains the `host`, `nslookup`, and `dig` tools that you can use to examine DNS records. When you run the `kubectl run` command, your terminal will be attached to the shell process running in the container (the `-it` option does this). When you exit the shell, the pod will be removed (by the `--rm` option).

NOTE Make sure you run the pod in the same namespace as the Kiada suite by using the `-n` option if required.

11.4.1 Inspecting a service's A and SRV records in DNS

You start by inspecting the `A` and `SRV` records associated with your services.

LOOKING UP A SERVICE'S A RECORD

To determine the IP address of the `quote` service, you run the `nslookup` command in the shell running in the container of the `dns-test` pod:

```
/ # nslookup quote
Server:      10.96.0.10
Address:     10.96.0.10# 53

Name:   quote.kiada.svc.cluster.local    #1
Address: 10.96.161.97                    #2
```

#1 The service's fully qualified domain name

#2 The service's cluster IP

NOTE You can use `dig` instead of `nslookup`, but you must either use the `+search` option or specify the fully qualified domain name of the service for the DNS lookup to succeed (run either `dig +search quote` or `dig quote.kiada.svc.cluster.local`).

Now look up the IP address of the `kiada` service. Although this service is of type `LoadBalancer` and thus has both an internal cluster IP and an external IP (that of the load balancer), the DNS returns only the cluster IP. This is to be expected since the DNS server is internal and is only used within the cluster.

LOOKING UP SRV RECORDS

A service provides one or more ports. Each port is given an `SRV` record in DNS. Use the following command to retrieve the `SRV` records for the `kiada` service:

```
/ # nslookup -query=SRV kiada
Server:      10.96.0.10
Address:     10.96.0.10# 53

kiada.kiada.svc.cluster.local    service = 0 50 80 kiada.kiada.svc.cluster.local.
kiada.kiada.svc.cluster.local    service = 0 50 443 kiada.kiada.svc.cluster.local.
```

#1 SRV record for the http port 80

#2 SRV record for the https port 443

NOTE As of this writing, GKE still runs kube-dns instead of CoreDNS. Kube-dns doesn't support all the DNS queries shown in this section.

A smart client running in a pod could look up the `SRV` records of a service to find out what ports are provided by the service. If you define the names for those ports in the Service object, they can even be looked up by name. The `SRV` record has the following form:

```
_port-name._port-protocol.service-name.namespace.svc.cluster.local
```

The names of the two ports in the `kiada` service are `http` and `https`, and both define TCP as the protocol. To get the `SRV` record for the `http` port, run the following command:

```

/ # nslookup -query=SRV _http._tcp.kiada
Server:      10.96.0.10
Address:     10.96.0.10# 53

_http._tcp.kiada.kiada.svc.cluster.local      service = 0 100 80 kiada.kiada.svc.

```

TIP To list all services and the ports they expose in the `kiada` namespace, you can run the command `nslookup -query=SRV any.kiada.svc.cluster.local`. To list all services in the cluster, use the name `any.any.svc.cluster.local`.

You'll probably never need to look for `SRV` records, but some Internet protocols, such as SIP and XMPP, depend on them to work.

NOTE Please leave the shell in the `dns-test` pod running, because you'll need it in the exercises in the next section when you learn about headless services.

11.4.2 Using headless services to connect to pods directly

Services expose a set of pods at a stable IP address. Each connection to that IP address is forwarded to a random pod or other endpoint that backs the service. Connections to the service are automatically distributed across its endpoints. But what if you want the client to do the load balancing? What if the client needs to decide which pod to connect to? Or what if it needs to connect to all pods that back the service? What if the pods that are part of a service all need to connect directly to each other? Connecting via the service's cluster IP clearly isn't the way to do this. What do you do in this situation?

Instead of connecting to the service IP, clients could get the pod IPs from the Kubernetes API, but it's better to keep them Kubernetes-agnostic and use standard mechanisms like DNS. Fortunately, you can configure the internal DNS to return the pod IPs instead of the service's cluster IP by creating a *headless* service.

For headless services, the cluster DNS returns not just a single `A` record pointing to the service's cluster IP, but multiple `A` records, one for each pod that's part of the service. Clients can therefore query the DNS to get the IPs of all the pods in the service. With this information, the client can then connect directly to the pods, as shown in figure 11.12.

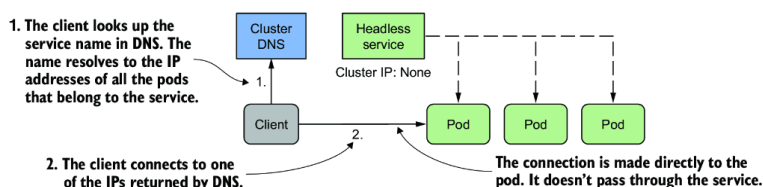


Figure 11.12 With headless services, clients connect directly to the pods.

CREATING A HEADLESS SERVICE

To create a headless service, you set the `clusterIP` field to `None`. Create another service for the quote pods but make this one headless. You can find the service manifest in the file `svc.quote-headless.yaml`. The following listing shows the contents of this file.

Listing 11.7 A headless service

```
apiVersion: v1
kind: Service
metadata:
  name: quote-headless
spec:
  clusterIP: None      #1
  selector:
    app: quote
  ports:
  - name: http
    port: 80
    targetPort: 80
    protocol: TCP
```

#1 Setting the clusterIP to None makes this a headless service.

After you create the service with `kubectl apply`, you can check it with

`kubectl get`. You'll see that it has no cluster IP:

```
$ kubectl get svc quote-headless -o wide
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	SELECTOR
quote-headless	ClusterIP	None	<none>	80/TCP	2m	app=quote

Because the service doesn't have a cluster IP, the DNS server can't return it when you try to resolve the service name. Instead, it returns the IP addresses of the pods. Before you continue, list the IPs of the pods that match the service's label selector as follows:

```
$ kubectl get po -l app=quote -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE
quote-canary	2/2	Running	0	3h	10.244.2.9	kind-worker2
quote-001	2/2	Running	0	3h	10.244.2.10	kind-worker2
quote-002	2/2	Running	0	3h	10.244.2.8	kind-worker2
quote-003	2/2	Running	0	3h	10.244.1.10	kind-worker

Note the IP addresses of these pods.

UNDERSTANDING DNS A RECORDS RETURNED FOR A HEADLESS SERVICE

To see what the DNS returns when you resolve the service, run the following command in the `dns-test` pod you created in the previous section:

```
/ # nslookup quote-headless
Server:      10.96.0.10
Address:     10.96.0.10# 53

Name:   quote-headless.kiada.svc.cluster.local
Address: 10.244.2.9                                #1
Name:   quote-headless.kiada.svc.cluster.local
Address: 10.244.2.8                                #2
Name:   quote-headless.kiada.svc.cluster.local
Address: 10.244.2.10                                #3
Name:   quote-headless.kiada.svc.cluster.local
Address: 10.244.1.10                                #4
```

#1 The IP of the quote-canary pod

#2 The IP of the quote-002 pod

#3 The IP of the quote-001 pod

#4 The IP of the quote-003 pod

The DNS server returns the IP addresses of the four pods that match the service's label selector. This is different from what DNS returns for regular (non-headless) services such as the `quote` service, where the name resolves to the cluster IP of the service:

```
/ # nslookup quote
Server:      10.96.0.10
Address:     10.96.0.10# 53

Name:   quote.kiada.svc.cluster.local
Address: 10.96.161.97      #1
```

#1 The cluster IP of the quote service

UNDERSTANDING HOW CLIENTS USE HEADLESS SERVICES

Clients that wish to connect directly to pods that are part of a service can do so by retrieving the `A` (or `AAAA`) records from the DNS. The client can then connect to one, some, or all the returned IP addresses.

Clients that don't perform the DNS lookup themselves can use the service as they'd use a regular, non-headless service. Because the DNS server rotates the list of IP addresses it returns, a client that simply uses the service's FQDN in the connection URL will get a different pod IP each time. Therefore, client requests are distributed across all pods.

You can try this by sending multiple requests the `quote-headless` service with `curl` from the `dns-test` pod as follows:

```
/ # while true; do curl http://quote-headless; done
This is the quote service running in pod quote-002
This is the quote service running in pod quote-001
This is the quote service running in pod quote-002
This is the quote service running in pod quote-canary
...
```

Each request is handled by a different pod, just like when you use the regular service. The difference is that with a headless service you connect directly to the pod IP, while with regular services you connect to the cluster IP of the service, and your connection is forwarded to one of the pods. You can see this by running `curl` with the `--verbose` option and examining the IP it connects to:

```
/ # curl --verbose http://quote-headless #1
* Trying 10.244.1.10:80... #1
* Connected to quote-headless (10.244.1.10) port 80 (#0)
...

/ # curl --verbose http://quote #2
* Trying 10.96.161.97:80... #2
* Connected to quote (10.96.161.97) port 80 (#0)
...
```

#1 When you connect to the headless service, you connect directly to one of the pods.

#2 When you connect to the regular service, you connect to its cluster IP.

HEADLESS SERVICES WITH NO LABEL SELECTOR

To conclude this section on headless services, I'd like to mention that services with manually configured endpoints (services without a label selector) can also be headless. If you omit the label selector and set the `clusterIP` to `None`, the DNS will return an `A`/`AAAA` record for each end-

point, just as it does when the service endpoints are pods. To test this yourself, apply the manifest in the `svc.external-service-headless.yaml` file and run the following command in the `dns-test` pod:

```
/ # nslookup external-service-headless
```

11.4.3 Creating a CNAME alias for an existing service

In the previous sections, you learned how to create `A` and `AAAA` records in the cluster DNS. To do this, you create Service objects that either specify a label selector to find the service endpoints, or you define them manually using the Endpoints and EndpointSlice objects.

There's also a way to add `CNAME` records to the cluster DNS. In Kubernetes, you add `CNAME` records to DNS by creating a Service object, just as you do for `A` and `AAAA` records.

NOTE A `CNAME` record is a DNS record that maps an alias to an existing DNS name, unlike an `A` record, which maps it to an IP address.

CREATING AN EXTERNALNAME SERVICE

To create a service that serves as an alias for an existing service, either an internal service or one that's external to the cluster, you create a Service object whose `type` field is set to `ExternalName`. The following listing shows an example of this type of service. You can find the manifest in the file `svc.time-api.yaml`.

Listing 11.8 An `ExternalName`-type service

```
apiVersion: v1
kind: Service
metadata:
  name: time-api
spec:
  type: ExternalName           #1
  externalName: worldtimeapi.org #2
```

#1 Service type is set to `ExternalName`.

#2 This is the fully qualified domain name that the `CNAME` record will point to.

In addition to setting the `type` to `ExternalName`, the service manifest must also specify in the `externalName` field external name to which this service resolves. No Endpoints or EndpointSlice object is required for `ExternalName` services.

CONNECTING TO AN EXTERNALNAME SERVICE FROM A POD

After the service is created, pods can connect to the external service using the domain name `time-api.<namespace>.svc.cluster.local` (or `time-api` if they're in the same namespace as the service) instead of using the actual FQDN of the external service, as shown in the following example:

```
$ kubectl exec -it kiada-001 -c kiada -- curl http://time-api/api/timezone/CET
```

RESOLVING EXTERNALNAME SERVICES IN DNS

Because `ExternalName` services are implemented at the DNS level (only a `CNAME` record is created for the service), clients don't connect to the service through the cluster IP, as is the case with non-headless ClusterIP

services. They connect directly to the external service. Like headless services, `ExternalName` services have no cluster IP, as the following output shows:

```
$ kubectl get svc time-api
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE	
time-api	ExternalName	<none>	worldtimeapi.org	80/TCP	4m51s	#1

#1 ExternalName services get no cluster IP.

As a final exercise in this section on DNS, you can try resolving the `time-api` service in the `dns-test` pod as follows:

```
/ # nslookup time-api
Server:      10.96.0.10
Address:     10.96.0.10# 53

time-api.kiada.svc.cluster.local      canonical name = worldtimeapi.org.  #1
Name:   worldtimeapi.org              #2
Address: 213.188.196.246              #2
Name:   worldtimeapi.org              #2
Address: 2a09:8280:1::3:e             #2
```

#1 The time-api service maps to worldtimeapi.org

#2 The address worldtimeapi.org resolves to an IPv4 and an IPv6 address.

You can see that `time-api.kiada.svc.cluster.local` points to `worldtimeapi.org`. This concludes this section on DNS records for Kubernetes services. You can now exit the shell in the `dns-test` pod by typing `exit` or pressing Ctrl-D. The pod is deleted automatically.

11.5 Configuring services to route traffic to nearby endpoints

When you deploy pods, they are distributed across the nodes in the cluster. If cluster nodes span different availability zones or regions and the pods deployed on those nodes exchange traffic with each other, network performance and traffic costs can become a problem. In this case, it makes sense for services to forward traffic to pods that aren't far from the pod where the traffic originates.

In other cases, a pod may need to communicate only with service endpoints on the same node as the pod—not for performance or cost reasons, but because only the node-local endpoints can provide the service in the proper context. Let me explain what I mean.

11.5.1 Forwarding traffic only within the same node with `internalTrafficPolicy`

If pods provide a service that's tied in some way to the node on which the pod is running, you must ensure that client pods running on a particular node connect only to the endpoints on the same node. You can do this by creating a Service with the `internalTrafficPolicy` set to `Local`.

NOTE You previously learned about the `externalTrafficPolicy` field, which is used to prevent unnecessary network hops between nodes when external traffic arrives in the cluster. The service's `internalTrafficPolicy` field is similar, but serves a different purpose.

As shown figure 11.13, if the service is configured with the `Local` internal traffic policy, traffic from pods on a given node is forwarded only to

Pods on the same node. If there are no node-local service endpoints, the connection fails.

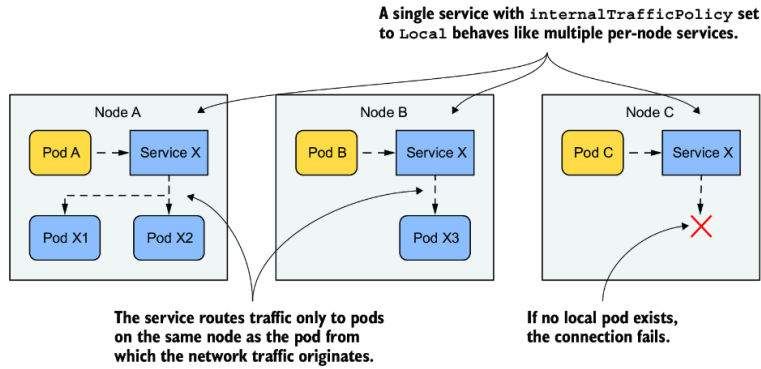


Figure 11.13 The behavior of a service with `internalTrafficPolicy` set to `Local`

Imagine a system pod running on each cluster node that manages communication with a device attached to the node. The pods don't use the device directly but communicate with the system pod. Since pod IPs are fungible, while service IPs are stable, pods connect to the system pod through a service. To ensure that pods connect only to the local system pod and not to those on other nodes, the service is configured to forward traffic only to local endpoints. You don't have any such pods in your cluster, but you can use the `quote` pods to try this feature.

CREATING A SERVICE WITH A LOCAL INTERNAL TRAFFIC POLICY

The following listing shows the manifest for a service named `quote-local`, which forwards traffic only to pods running on the same node as the client pod. You can find the manifest in the file `svc.quote-local.yaml`.

Listing 11.9 A service that only forwards traffic to local endpoints

```
apiVersion: v1
kind: Service
metadata:
  name: quote-local
spec:
  internalTrafficPolicy: Local      #1
  selector:
    app: quote
  ports:
  - name: http
    port: 80
    targetPort: 80
    protocol: TCP
```

#1 This service forwards traffic from pods only to endpoints on the same node as the pod.

As you can see in the manifest, the service will forward traffic to all pods with the label `app: quote`, but since `internalTrafficPolicy` is set to `Local`, it won't forward traffic to all `quote` pods in the cluster, but only to the pods that are on the same node as the client pod. Create the service by applying the manifest with `kubectl apply`.

OBSERVING NODE-LOCAL TRAFFIC ROUTING

Before you can see how the service routes traffic, you need to figure out where the client pods and the pods that are the endpoints of the service are located. List the pods with the `-o wide` option to see which node each pod is running on.

Select one of the `kiada` pods and note its cluster node. Use `curl` to connect to the `quote-local` service from that pod. For example, my `kiada-001` pod runs on the `kind-worker` node. If I run `curl` in it multiple times, all requests are handled by the quote pods on the same node:

```
$ kubectl exec kiada-001 -c kiada -- sh -c "while :; do curl -s quote-local; done"
This is the quote service running in pod quote-002 on node kind-worker      #1
This is the quote service running in pod quote-canary on node kind-worker  #1
This is the quote service running in pod quote-canary on node kind-worker  #1
This is the quote service running in pod quote-002 on node kind-worker      #1
```

#1 Both pods run on the same node as the `kiada-001` pod.

No request is forwarded to the pods on the other node(s). If I delete the two pods on the `kind-worker` node, the next connection attempt will fail:

```
$ kubectl exec -it kiada-001 -c kiada -- curl http://quote-local
curl: (7) Failed to connect to quote-local port 80: Connection refused
```

In this section, you learned how to forward traffic only to node-local endpoints when the semantics of the service require it. In other cases, you may want traffic to be forwarded preferentially to endpoints near the client pod, and only to more distant pods when needed. You'll learn how to do this in the next section.

11.5.2 Topology-aware hints

Imagine the Kiada suite running in a cluster with nodes spread across multiple data centers in different zones and regions, as shown in figure 11.14. You don't want a Kiada pod running in one zone to connect to Quote pods in another zone, unless there are no Quote pods in the local zone. Ideally, you want connections to be made within the same zone to reduce network traffic and associated costs.

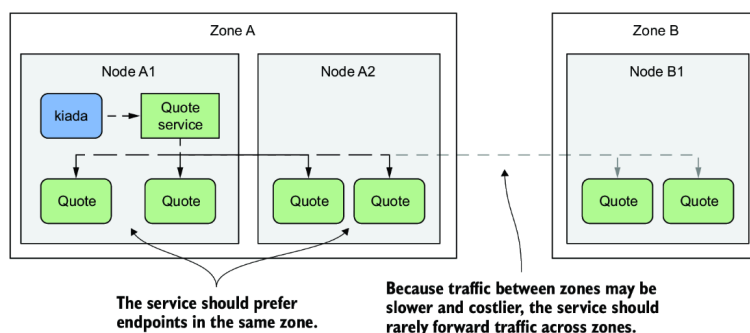


Figure 11.14 Routing serviced traffic across availability zones

The figure illustrates what is called *topology-aware traffic routing*. Kubernetes supports it by adding topology-aware hints to each endpoint in the `EndpointSlice` object.

UNDERSTANDING HOW TOPOLOGY AWARE HINTS ARE CALCULATED

First, all your cluster nodes must contain the `kubernetes.io/zone` label to indicate which zone each node is located in. To indicate that a service should use topology-aware hints, you must set the `service.kubernetes.io/topology-aware-hints` annotation to `Auto`. If the service has a sufficient number of endpoints, Kubernetes adds the hints to each endpoint in the `EndpointSlice` object(s). As you can see in the following listing, the `hints` field specifies the zones from which this endpoint is to be consumed.

Listing 11.10 EndpointSlice with topology aware hints

```
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
endpoints:
- addresses:
  - 10.244.2.2
  conditions:
    ready: true
  hints:
    #1
    forZones:
    #1
    - name: zoneA
    #1
  nodeName: kind-worker
  targetRef:
    kind: Pod
    name: quote-002
    namespace: default
    resourceVersion: "944"
    uid: 03343161-971d-403c-89ae-9632e7cd0d8d
  zone: zoneA
  #2
...

```

#1 This endpoint should be consumed by clients running in zoneA.

#2 This endpoint is located in zoneA.

The listing shows only a single endpoint. The endpoint represents the pod `quote-002` running on node `kind-worker`, which is located in `zoneA`. For this reason, the `hints` for this endpoint indicate that it is to be consumed by pods in `zoneA`. In this particular case, only `zoneA` should use this endpoint, but the `forZones` array could contain multiple zones.

These hints are computed by the EndpointSlice controller, which is part of the Kubernetes control plane. It assigns endpoints to each zone based on the number of CPU cores that can be allocated in the zone. If a zone has a higher number of CPU cores, it'll be assigned a higher number of endpoints than a zone with fewer CPU cores. In most cases, the hints ensure that traffic is kept within a zone, but to ensure a more even distribution, this isn't always the case.

UNDERSTANDING WHERE TOPOLOGY AWARE HINTS ARE USED

Each node ensures that traffic sent to the service's cluster IP is forwarded to one of the service's endpoints. If there are no topology-aware hints in the EndpointSlice object, all endpoints, regardless of the node on which they reside, will receive traffic originating from a particular node. However, if all endpoints in the EndpointSlice object contain hints, each node processes only the endpoints that contain the node's zone in the hints and ignores the rest. Traffic originating from a pod on the node is therefore forwarded to only some endpoints.

Currently, you can't influence topology-aware routing except to turn it on or off, but that may change in the future.

11.6 Managing the inclusion of a pod in service endpoints

There's one more thing about services and endpoints that wasn't covered yet. You learned that a pod is included as an endpoint of a service if its labels match the service's label selector. Once a new pod with matching labels shows up, it becomes part of the service and connections are forwarded to the pod. But what if the application in the pod isn't immediately ready to accept connections?

It may be that the application needs time to load either the configuration or the data, or that it needs to warm up so that the first client connection can be processed as quickly as possible without unnecessary latency caused by the fact that the application has just started. In such cases, you don't want the pod to receive traffic immediately, especially if the existing pod instances can handle the traffic. It makes sense not to forward requests to a pod that's just starting up until it becomes ready.

11.6.1 Introducing readiness probes

In chapter 6, you learned how to keep your applications healthy by letting Kubernetes restart containers that fail their liveness probes. A similar mechanism called *readiness probes* allows an application to signal that it's ready to accept connections.

Like liveness probes, the Kubelet also calls the readiness probe periodically to determine the readiness status of the pod. If the probe is successful, the pod is considered ready. The opposite is true if it fails. Unlike liveness probes, a container whose readiness probe fails isn't restarted; it's only removed as an endpoint from the services to which it belongs.

As shown figure 11.15, if a pod fails its readiness probe, the service doesn't forward connections to the pod even though its labels match the label selector defined in the service.

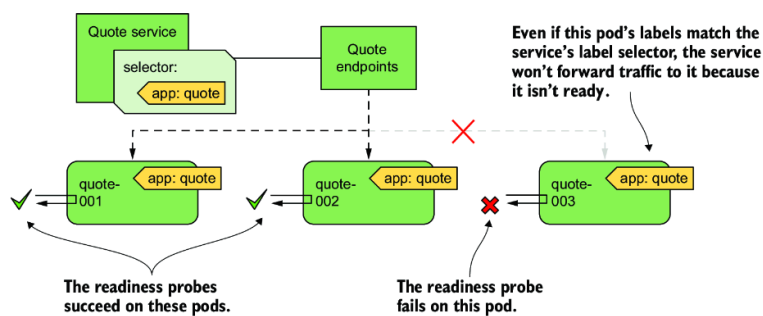


Figure 11.15 Pods that fail the readiness probe are removed from the service.

The notion of being ready is specific to each application. The application developer decides what readiness means in the context of their application. To do this, they expose an endpoint through which Kubernetes asks the application whether it's ready. Depending on the type of endpoint, the correct readiness probe type must be used.

UNDERSTANDING READINESS PROBE TYPES

As with liveness probes, Kubernetes supports three types of readiness probes:

- An `exec` probe executes a process in the container. The exit code used to terminate the process determines whether the container is ready.
- An `httpGet` probe sends a `GET` request to the container via HTTP or HTTPS. The response code determines the container's readiness status.
- A `tcpSocket` probe opens a TCP connection to a specified port on the container. If the connection is established, the container is considered ready.

CONFIGURING HOW OFTEN THE PROBE IS EXECUTED

You may recall that you can configure when and how often the liveness probe runs for a given container using the following properties: `initialDelaySeconds`, `periodSeconds`, `failureThreshold`, and `time-`

outSeconds . These properties also apply to readiness probes, but they also support the additional `successThreshold` property, which specifies how many times the probe must succeed for the container to be considered ready.

These settings are best explained graphically. Figure 11.16 shows how the individual properties affect the execution of the readiness probe and the resulting readiness status of the container.

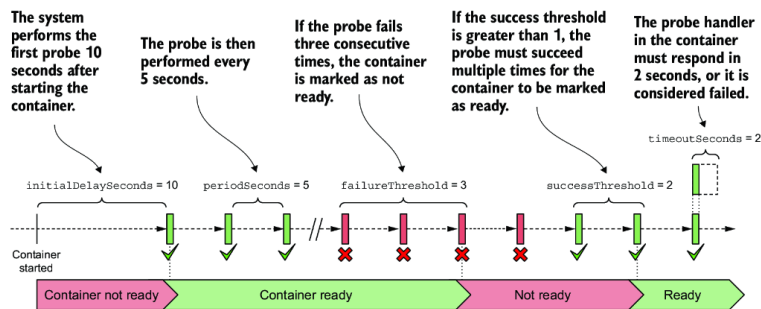


Figure 11.16 Readiness probe execution and resulting readiness status of the container

NOTE If the container defines a startup probe, the initial delay for the readiness probe begins when the startup probe succeeds. Startup probes are explained in chapter 6.

When the container is ready, the pod becomes an endpoint of the services whose label selector it matches. When it's no longer ready, it's removed from those services.

11.6.2 Adding a readiness probe to a pod

To see readiness probes in action, create a new pod with a probe that you can switch from success to failure at will. This isn't a real-world example of how to configure a readiness probe, but it allows you to see how the outcome of the probe affects the pod's inclusion in the service.

The following listing shows the relevant part of the pod manifest file `pod.kiada-mock-readiness.yaml`, which you can find in the book's code repository.

Listing 11.11 A readiness probe definition in a pod

```
apiVersion: v1
kind: Pod
...
spec:
  containers:
  - name: kiada
    ...
    readinessProbe:      #1
      exec:              #2
        command:         #2
        - ls             #2
        - /var/ready     #2
      initialDelaySeconds: 10    #3
      periodSeconds: 5          #3
      failureThreshold: 3       #3
      successThreshold: 2       #3
      timeoutSeconds: 2         #3
    ... #3
```

#1 A readiness probe is defined for the kiada container.

#2 The probe executes the `ls` command in the container.

#3 Defines when and how often the probe is executed and how many times it must fail or succeed for the container's readiness state to change. It also sets the timeout for each invocation of the probe.

The readiness probe periodically runs the `ls /var/ready` command in the `kiada` container. The `ls` command returns the exit code zero if the file exists, otherwise it's nonzero. Since zero is considered a success, the readiness probe succeeds if the file is present.

The reason to define such a strange readiness probe is so that you can change its outcome by creating or removing the file in question. When you create the pod, the file doesn't exist yet, so the pod isn't ready. Before you create the pod, delete all other `kiada` pods except `kiada-001`. This makes it easier to see the service endpoints change.

OBSERVING THE PODS' READINESS STATUS

After you create the pod from the manifest file, check its status as follows:

```
$ kubectl get po kiada-mock-readiness
```

NAME	READY	STATUS	RESTARTS	AGE	
kiada-mock-readiness	1/2	Running	0	1m	#1

#1 Only one of the pod's containers is ready.

The `READY` column shows that only one of the pod's containers is ready. This is the `envoy` container, which doesn't define a readiness probe. Containers without a readiness probe are considered ready as soon as they're started.

Since the pod's containers aren't all ready, the pod shouldn't receive traffic sent to the service. You can check this by sending several requests to the Kiada service. You'll notice that all requests are handled by the `kiada-001` pod, which is the only active endpoint of the service. This is evident from the `Endpoints` and `EndpointSlice` objects associated with the service. For example, the `kiada-mock-readiness` pod appears in the `notReadyAddresses` instead of the `addresses` array in the `Endpoints` object:

```
$ kubectl get endpoints kiada -o yaml
apiVersion: v1
kind: Endpoints
metadata:
  name: kiada
  ...
subsets:
- addresses:
  - ...
  notReadyAddresses: #1
- ip: 10.244.1.36 #1
  nodeName: kind-worker2 #1
  targetRef: #1
    kind: Pod #1
    name: kiada-mock-readiness #1
    namespace: default #1
  ... #1
```

#1 The kiada-mock-readiness pod appears among the service's

notReadyAddresses.

In the EndpointSlice object, the endpoint's `ready` condition is `false`:

```
$ kubectl get endpointslices -l kubernetes.io/service-name=kiada -o yaml
apiVersion: v1
items:
- addressType: IPv4
  apiVersion: discovery.k8s.io/v1
  endpoints:
  - addresses:
    - 10.244.1.36
    conditions: #1
      ready: false #1
    nodeName: kind-worker2
    targetRef:
      kind: Pod
      name: kiada-mock-readiness
      namespace: default
    ...
```

#1 The kiada-mock-readiness pod's ready condition is false.

NOTE In some cases, you may want to disregard the readiness status of pods. This may be the case if you want all pods in a group to get `A`, `AAAA`, and `SRV` records even though they aren't ready. If you set the `publishNotReadyAddresses` field in the Service object's spec to `true`, non-ready pods are marked as ready in both the Endpoints and EndpointSlice objects. Components like the cluster DNS treat them as ready.

For the readiness probe to succeed, create the `/var/ready` file in the container as follows:

```
$ kubectl exec kiada-mock-readiness -c kiada -- touch /var/ready
```

The `kubectl exec` command runs the `touch` command in the `kiada` container of the `kiada-mock-readiness` pod. The `touch` command creates the specified file. The container's readiness probe will now be successful. All the pod's containers should now show as ready. Verify that this is the case as follows:

```
$ kubectl get po kiada-mock-readiness
```

NAME	READY	STATUS	RESTARTS	AGE
kiada-mock-readiness	1/2	Running	0	10m

Surprisingly, the pod is still not ready. Is something wrong or is this the expected result? Take a closer look at the pod with `kubectl describe`. In the output, you'll find the following line:

```
Readiness:  exec [ls /var/ready] delay=10s timeout=2s period=5s #success=2 #failur
```

The readiness probe defined in the pod is configured to check the status of the container every 5 seconds. However, it's also configured to require two consecutive probe attempts to be successful before setting the status of the container to `ready`. Therefore, it takes about 10 seconds for the pod to be ready after you create the `/var/ready` file.

When this happens, the pod should become an active endpoint of the service. You can verify this is the case by examining the Endpoints or EndpointSlice objects associated with the service, or by simply accessing the service a few times and checking to see whether the `kiada-mock-readiness` pod receives any of the requests you send.

If you want to remove the pod from the service again, run the following command to remove the `/var/ready` file from the container:

```
$ kubectl exec kiada-mock-readiness -c kiada -- rm /var/ready
```

This mockup of a readiness probe is just to show how readiness probes work. In the real world, the readiness probe shouldn't be implemented in this way. If you want to manually remove pods from a service, you can do so by either deleting the pod or changing the pod's labels rather than manipulating the readiness probe outcome.

TIP If you want to manually control whether a pod is included in a service, add a label key such as `enabled` to the pod and set its value to `true`. Then add the label selector `enabled=true` to your service. Remove the label from the pod to remove the pod from the service.

11.6.3 Implementing real-world readiness probes

If you don't define a readiness probe in your pod, it becomes a service endpoint as soon as it's created. This means that every time you create a new pod instance, connections forwarded by the service to that new instance will fail until the application in the pod is ready to accept them. To prevent this, you should always define a readiness probe for the pod.

In the previous section, you learned how to add a mock readiness probe to a container to manually control whether the pod is a service endpoint. In the real world, the readiness probe result should reflect the ability of the application running in the container to accept connections.

DEFINING A MINIMAL READINESS PROBE

For containers running an HTTP server, it's much better to define a simple readiness probe that checks whether the server responds to a simple `GET /` request, such as the one in the following code snippet, than to have no readiness probe at all:

```

readinessProbe:
  httpGet:
    port: 8080
    path: /
    scheme: HTTP

```

#1 The probe sends an HTTP GET request to port 8080 of the container.

#2 The probe requests the root URL path over HTTP (as opposed to HTTPS).

When Kubernetes invokes this readiness probe, it sends the `GET /` request to port `8080` of the container and checks the returned HTTP response code. If the response code is greater than or equal to `200` and less than `400`, the probe is successful, and the pod is considered ready. If the response code is anything else (for example, `404` or `500`) or the connection attempt fails, the readiness probe is considered failed and the pod is marked as not ready.

This simple probe ensures that the pod only becomes part of the service when it can actually handle HTTP requests, rather than immediately when the pod is started.

DEFINING A BETTER READINESS PROBE

A simple readiness probe like the one shown in the previous section isn't always sufficient. Take the Quote pod, for example. You may recall that it runs two containers. The `quote-writer` container selects a random quote from this book and writes it to a file called `quote` in the volume shared by the two containers. The `nginx` container serves files from this shared volume. Thus, the quote itself is available at the URL path `/quote`.

The purpose of the Quote pod is clearly to provide a random quote from the book. Therefore, it shouldn't be marked ready until it can serve this quote. If you direct the readiness probe to the URL path `/`, it'll succeed even if the `quote-writer` container hasn't yet created the `quote` file. Therefore, the readiness probe in the Quote pod should be configured as shown in the following snippet from the `pod.quote-readiness.yaml` file:

```

readinessProbe:
  httpGet:
    port: 80
    path: /quote
    scheme: HTTP
  failureThreshold: 1

```

#1 The Quote pod is ready when it can serve the quote.

#2 Set the failure threshold to one, so that the pod is immediately marked as not ready if the probe fails.

If you add this readiness probe to your Quote pod, you'll see that the pod is only ready when the `quote` file exists. Try deleting the file from the pod as follows:

```
$ kubectl exec quote-readiness -c quote-writer -- rm /var/local/output/quote
```

Now check the pod's readiness status with `kubectl get pod`, and you'll see that one of the containers is no longer ready. When the `quote-writer` recreates the file, the container becomes ready again. You can also inspect the endpoints of the `quote` service with `kubectl get endpoints quote` to see that the pod is removed and then re-added.

IMPLEMENTING A DEDICATED READINESS ENDPOINT

As you saw in the previous example, it may be sufficient to point the readiness probe to an existing path served by the HTTP server, but it's also common for an application to provide a dedicated endpoint, such as `/healthz/ready` or `/readyz`, through which it reports its readiness status. When the application receives a request on this endpoint, it can perform a series of internal checks to determine its readiness status.

Let's take the Quiz service as an example. The Quiz pod runs both an HTTP server and a MongoDB container. As you can see in the following listing, the `quiz-api` server implements the `/healthz/ready` endpoint. When it receives a request, it checks if it can successfully connect to MongoDB in the other container. If so, it responds with a `200 OK`. If not, it returns `500 Internal Server Error`.

Listing 11.12 The readiness endpoint in the quiz-api application

```
func (s *HTTPServer) ListenAndServe(listenAddress string) {
    router := mux.NewRouter()
    router.Methods("GET").Path("/").HandlerFunc(s.handleRoot)
    router.Methods("GET").Path("/healthz/ready").HandlerFunc(s.handleReadiness) #1
    ...
}

func (s *HTTPServer) handleReadiness(res http.ResponseWriter, req *http.Request) {
    conn, err := s.db.Connect() #2
    if err != nil { #3
        res.WriteHeader(http.StatusInternalServerError) #3
        _, _ = fmt.Fprintf(res, "ERROR: %v\n", err.Error()) #3
        return #3
    }
    defer conn.Close()

    res.WriteHeader(http.StatusOK) #4
    _, _ = res.Write([]byte("Readiness check successful")) #4
}
```

#1 The `/healthz/ready` endpoint invokes the `handleReadiness()` function.

#2 Tries to connect to MongoDB

#3 If the connection fails, the `500 Internal Server Error` response code is returned.

#4 If the connection succeeds, the `200 OK` response code is returned.

The readiness probe defined in the Quiz pod ensures that everything the pod needs to provide its services is present and working. As additional components are added to the `quiz-api` application, further checks can be added to the readiness check code. An example is the addition of an internal cache. The readiness endpoint could check whether the cache is warmed up so that the pod is only then exposed to clients.

CHECKING DEPENDENCIES IN THE READINESS PROBE

In the Quiz pod, the MongoDB database is an internal dependency of the `quiz-api` container. The Kiada pod, on the other hand, depends on the Quiz and Quote services, which are external dependencies. What should the readiness probe check in the Kiada pod? Should it check whether it can reach the Quote and Quiz services?

The answer to this question is debatable, but any time you check dependencies in a readiness probe, you must consider what happens if a transient problem, such as a temporary increase in network latency, causes the probe to fail.

Note that the `timeoutSeconds` field in the readiness probe definition limits the time the probe has to respond. The default timeout is only 1 second. The container must respond to the readiness probe within this time.

If the Kiada pod calls the other two services in its readiness check, but their responses are only slightly delayed due to a transient network disruption, its readiness probe fails and the pod is removed from the service endpoints. If this happens to all Kiada pods at the same time, there will be no pods left to handle client requests. The disruption may only last a second, but the pods may not be added back to the service until dozens of seconds later, depending on how the `periodSeconds` and `successThreshold` properties are configured.

When you check external dependencies in your readiness probes, you should consider what happens when these types of transient network problems occur. Then you should set your periods, timeouts, and thresholds accordingly.

TIP Readiness probes that try to be too smart can cause more problems than they solve. As a rule of thumb, readiness probes shouldn't test external dependencies but can test dependencies within the same pod.

The Kiada application also implements the `/healthz/ready` endpoint instead of having the readiness probe use the `/` endpoint to check its status. This endpoint simply responds with the HTTP response code `200 OK` and the word `Ready` in the response body, which ensures that the readiness probe only checks that the application itself is responding, without also connecting to the Quiz or Quote services. You can find the pod manifest in the `pod.kiada-readiness.yaml` file.

UNDERSTANDING READINESS PROBES IN THE CONTEXT OF POD SHUTDOWN

One last note before we close this chapter. As you know, readiness probes are most important when the pod starts, but they also ensure that the pod is taken out of service when something causes it to no longer be ready during normal operation. But what about when the pod is terminating? A pod that's in the process of shutting down shouldn't be part of any services. Do you need to consider that when implementing the readiness probe?

Fortunately, when you delete a pod, Kubernetes not only sends the termination signal to the pod's containers, but also removes the pod from all services, which means you don't have to make any special provisions for terminating pods in your readiness probes. You don't have to make sure that the probe fails when your application receives the termination signal.

Summary

- Pods communicate over a flat network that allows any pod to reach any other pod in the cluster, regardless of the actual network topology connecting the cluster nodes.
- A Kubernetes service makes a group of pods available under a single IP address. While the IPs of the pods may change, the IP of the service remains constant.
- The cluster IP of the service is reachable from inside the cluster, but `NodePort` and `LoadBalancer` services are also accessible from outside the cluster.

- Service endpoints are either determined by a label selector specified in the Service object or configured manually. These endpoints are stored in the Endpoints and EndpointSlice objects.
- Client pods can find services using the cluster DNS or environment variables. Depending on the type of Service, the following DNS records may be created: `A` , `AAAA` , `SRV` , and `CNAME` .
- Services can be configured to forward external traffic only to pods on the same node that received the external traffic, or to pods anywhere in the cluster. They can also be configured to route internal traffic only to pods on the same node as the pod from which the traffic originates from. Topology-aware routing ensures that traffic isn't routed across availability zones when a local pod can provide the requested service.
- Pods don't become service endpoints until they're ready. By implementing a readiness probe handler in an application, you can define what readiness means in the context of that particular application.